

SECURITY ASSESSMENT REPORT

User Management Web Application

Application	OWASP NodeGoat (RetireEasy)
Assessment Type	Week 1 – Security Assessment
Tester	Fatima Haq
Date	June 26, 2026
Risk Rating	HIGH
Total Findings	14 Alerts (8 Vulnerabilities + Informational)

CONFIDENTIAL – For Educational / Internship Use Only

1. Objective

The objective of this assessment was to analyze a web application for common security vulnerabilities and identify areas where security can be improved. OWASP NodeGoat (RetireEasy) was used as the target application for testing. All testing was performed in a controlled, local environment on an intentionally vulnerable application designed for educational purposes.Used **http://localhost:4000/**

2. Testing Environment

Application	OWASP NodeGoat (RetireEasy)
Platform	Localhost
Browser	Google Chrome
Scanner	OWASP ZAP 2.17.0
Database	MongoDB 4.4.7
Runtime	Node.js

3. Security Assessment Activities

3.1 Application Exploration

The application was successfully installed and configured. The following functionalities were explored:

- User Registration (Sign Up)
- User Login
- User Profile
- Memos Section
- Dashboard and Account Features

3.2 Cross-Site Scripting (XSS) Testing

The following payload was entered into application input fields during manual testing:

```
<script>alert('XSS')</script>
```

Testing was performed on profile fields and memo sections. The input was stored and reflected back in the browser without sanitization, causing the script to execute. OWASP ZAP automated scanning independently confirmed Reflected XSS on multiple endpoints.

Result	Vulnerability Confirmed – Script Executed in Browser
Risk Level	HIGH
OWASP Category	A03:2021 – Injection

Impact:

- Execution of malicious JavaScript in victims' browsers

- Session hijacking and cookie theft
- User impersonation and account takeover
- Phishing redirects and UI defacement

Recommendations:

- Validate and sanitize all user-supplied input on the server side
- Encode output before rendering (use context-aware escaping)
- Implement a strict Content Security Policy (CSP) header
- Use templating engines that auto-escape output (e.g., EJS <%= %> vs <%- %>)

3.3 SQL Injection Testing

The following payload was tested in the login form:

```
admin' OR '1'='1
```

The application rejected the input and authentication bypass was not achieved. This indicates the application likely uses parameterized queries or MongoDB's query structure for authentication, which provides inherent protection against classic SQL injection. However, NoSQL injection variants should be assessed in future testing phases.

Result	Not Detected During Testing
Risk Level	Low (SQL) – NoSQL Injection not yet assessed

Recommendations:

- Continue using parameterized database queries or ODM libraries (e.g., Mongoose)
- Assess for NoSQL injection vulnerabilities (e.g., \$where, \$regex abuse in MongoDB)
- Validate and sanitize all user input regardless of database type

3.4 Weak Password Storage Assessment

Review of the application source code revealed that user passwords are stored directly in the database without hashing or salting:

```
password: password
```

The codebase contained commented-out bcrypt hashing code, indicating awareness of the issue but lack of implementation. If the database is compromised, all user passwords would be immediately exposed in plain text.

Result	Vulnerability Confirmed – Plain Text Storage
Risk Level	HIGH
OWASP Category	A02:2021 – Cryptographic Failures

Impact:

- Immediate password disclosure if the database is breached
- Credential stuffing attacks on other services (password reuse)
- Violation of data protection regulations (GDPR, etc.)

Recommendations:

- Store passwords using bcrypt with a minimum cost factor of 12
- Never store, log, or transmit passwords in plain text
- Enforce a strong password policy (minimum length, complexity)

3.5 Security Misconfiguration Assessment

MongoDB displayed the following warning during application startup:

"Access control is not enabled for the database."

This means anyone with network access to the database port can read, modify, or delete all data without providing credentials. This is a critical misconfiguration in any environment beyond local development.

Result	Vulnerability Confirmed – No Database Authentication
Risk Level	Medium
OWASP Category	A05:2021 – Security Misconfiguration

Impact:

- Unauthorized read/write/delete access to all database records
- Exposure of all user credentials, personal data, and session tokens
- Potential for complete data loss or ransomware attack

Recommendations:

- Enable MongoDB authentication and create role-based users
- Restrict database port access with firewall rules
- Never expose database ports to the public internet
- Follow MongoDB Security Checklist for production hardening

4. OWASP ZAP Automated Scan Results

OWASP ZAP version 2.17.0 performed an automated scan against the running application at http://localhost. A total of 14 alerts were identified across 4 risk categories.

Risk Level	Alert Description
High	Cross-Site Scripting (Reflected)
Medium	CSP: Failure to Define Directive With No Fallback
Medium	Content Security Policy (CSP) Header Not Set
Medium	Missing Anti-Clickjacking Header
Medium	Vulnerable JavaScript Library
Low	Cookie Without SameSite Attribute

Risk Level	Alert Description
Low	Cross-Domain JavaScript Source File Inclusion
Low	Server Leaks Information via X-Powered-By Header
Low	X-Content-Type-Options Header Missing
Info	Authentication Request Identified
Info	Information Disclosure – Suspicious Comments
Info	Modern Web Application Identified
Info	Session Management Response Identified
Info	User Controllable HTML Element Attribute (Potential XSS)

5. Summary of Findings

Vulnerability	Severity	Status
Cross-Site Scripting (XSS)	High	Confirmed
Weak Password Storage	High	Confirmed
Security Misconfiguration (MongoDB)	Medium	Confirmed
Missing CSP Header	Medium	Confirmed
Missing Anti-Clickjacking Header	Medium	Confirmed
Vulnerable JavaScript Libraries	Medium	Confirmed
Cookie Without SameSite Attribute	Low	Confirmed
Information Leakage via Headers	Low	Confirmed
SQL Injection	Not Detected	Tested – Not Exploited

6. Review Notes & Improvements to Original Report

The following improvements were made during this review:

- Weak Password Storage severity upgraded from Medium to HIGH — plain text password storage is a critical risk directly enabling credential theft after any database breach.
- SQL Injection section expanded to mention NoSQL injection as a relevant future test vector, given the MongoDB backend.
- Impact sections added for all findings to clearly express business risk.
- OWASP category references (2021 Top 10) added to each finding for industry alignment.
- Testing environment table added to document exact tool versions.
- Overall report structure formalized with numbered sections and professional formatting.

7. Conclusion

The security assessment identified several vulnerabilities and security weaknesses within the OWASP NodeGoat application. The most critical findings are Cross-Site Scripting (XSS) and plain text password storage, both of which pose immediate risk to user account security. Security misconfigurations including the absence of MongoDB authentication and missing HTTP security headers further increase the attack surface.

Implementing the recommended controls — input validation, output encoding, bcrypt password hashing, Content Security Policy, and database access controls — would significantly reduce the risk exposure of this application. These findings provide a strong foundation for Week 2 remediation activities.

This report was produced as part of a cybersecurity internship training exercise. All testing was performed on a deliberately vulnerable application in a local environment. No real systems were tested.